

Fase 4: File Uploads

S3 Presigned URLs + Async Processing

Tutorial paso a paso

Proyecto Sentinel AcademIA
lFunknown

20 de junio de 2026

¿Qué construimos en este tutorial?

En la **Fase 4** permitimos que los usuarios suban **archivos de evidencia** (fotos, PDFs) a sus quejas. Esto se hace con el patrón de **presigned URLs**:

1. Cliente crea queja (POST /api/quejas) – ya existía (Fase 1)
2. Cliente pide URL de upload (POST /api/quejas/{id}/upload-url)
3. API devuelve URL temporal (válida 15 min)
4. Cliente sube el archivo **directamente a S3** (sin pasar por la API)
5. S3 dispara un evento que llama a **file_processor** Lambda
6. Lambda actualiza la queja en DynamoDB con la URL del archivo

Decisiones clave:

- **Por qué presigned URLs:** el cliente sube directo a S3, evitando saturar la API
- **Por qué S3 event:** desacopla el upload del procesamiento
- **Qué se logró:** end-to-end upload funcionando (POST → URL → S3 PUT → Lambda → DynamoDB → GET con adjuntos)

Email (SES) – no implementado en esta versión

La parte de notificar por email (SES) **no se implementó** en esta versión. El `analyze_queja` actualiza el status a `ANALIZADA` pero no envía email. Para prod: implementar `notifyByEmail` con AWS SES.

Índice

1. El patrón de presigned URLs	3
1.1. ¿Qué es una presigned URL?	3
1.2. ¿Por qué este patrón?	3
1.3. Diagrama del flujo	3
2. Componente 1: S3 Bucket	3
2.1. Crear el bucket en SAM	3
3. Componente 2: file_service.py (presigned URLs)	4
3.1. Función principal: <code>generate_presigned_upload_url</code>	4

4. Componente 3: upload_url Lambda	5
4.1. El handler	5
5. Componente 4: file_processor Lambda (S3 event consumer)	6
5.1. El handler	6
6. Componente 5: SAM template (los Lambdas nuevos)	7
6.1. Upload URL Lambda + permisos	7
7. El problema: circular dependency	8
7.1. El bug que enfrentamos	8
7.2. ¿Por qué?	8
7.3. La solución: configurar S3 notification manual	8
8. Los 2 bugs que arreglamos	9
9. El test E2E (paso a paso con outputs)	10
9.1. Paso 1: Crear queja	10
9.2. Paso 2: Pedir upload URL	10
9.3. Paso 3: Subir archivo a S3 (directo, sin API)	10
9.4. Paso 4: S3 event → file_processor → DynamoDB	11
10.Verificación en AWS Console	11
10.1. Recursos desplegados	11
10.2. Ver logs del file_processor	11
11.Catálogo de errores y soluciones	11
12.Lo que aprendimos (resumen ejecutivo)	12
13.Ejercicios para practicar	12
13.1. Ejercicio 1: GET /api/quejas/{id}/files	12
13.2. Ejercicio 2: DELETE /api/quejas/{id}/files/{key}	12
13.3. Ejercicio 3: Image thumbnails	13
13.4. Ejercicio 4: Email notification (Fase 4 parte 2)	13
13.5. Ejercicio 5: Antivirus scan	13
14.Siguiente paso: Fase 5 (RAG)	13

1. El patrón de presigned URLs

1.1. ¿Qué es una presigned URL?

Una URL temporal que **S3 genera** y que permite a un cliente **subir archivos directamente a S3** sin tener credenciales de AWS.

Características:

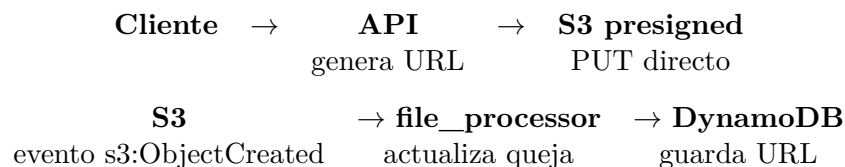
- ✓ **Temporal**: expira (configurable, default 15 min)
- ✓ **Específica**: solo PUT (o GET) sobre UN key específico
- ✓ **Scoped**: puede limitar ContentType, ContentLength
- ✓ **No requiere credenciales** del cliente

1.2. ¿Por qué este patrón?

Alternativa naive: el cliente envía el archivo a la API, la API lo sube a S3.

- ✗ **Problema 1**: la API recibe archivos grandes (10MB+), la Lambda tiene límite de 6MB de payload (sincrónico) o 250MB (async)
- ✗ **Problema 2**: desperdicia bandwidth y CPU de la Lambda
- ✗ **Problema 3**: timeout si el upload es lento
- ✓ **Solución con presigned URL**: la API solo genera la URL, el cliente sube directo

1.3. Diagrama del flujo



2. Componente 1: S3 Bucket

2.1. Crear el bucket en SAM

```

1 FilesBucket:
2   Type: AWS::S3::Bucket
3   Properties:
4     BucketName: !Sub "sentinel-files-${Stage}-${AWS::AccountId}"
5     PublicAccessBlockConfiguration:
6       BlockPublicAcls: true
7       BlockPublicPolicy: true
8       IgnorePublicAcls: true
9       RestrictPublicBuckets: true
10    CorsConfiguration:
11      CorsRules:
12        - AllowedOrigins: ["*"]
13          AllowedMethods: [GET, PUT, POST]
14          AllowedHeaders: ["*"]
15          MaxAge: 3000

```

Listing 1: infra/template.yaml S3 bucket

Decisiones:

- **✓Nombre único:** incluye account ID para evitar colisiones
- **✓PublicAccessBlock: todo ON:** el bucket NO es público
- **✓Acceso vía presigned URL:** solo usuarios autorizados
- **✓CORS:** permite PUT desde el frontend

3. Componente 2: file_service.py (presigned URLs)**3.1. Función principal: generate_presigned_upload_url**

```

1 import boto3
2 import re
3 import uuid
4
5 ALLOWED_CONTENT_TYPES = [
6     "image/jpeg", "image/png", "image/gif", "image/webp",
7     "application/pdf",
8     "application/msword",
9     "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
10    "text/plain",
11 ]
12 MAX_FILE_SIZE_BYTES = 10 * 1024 * 1024 # 10 MB
13
14 def build_s3_key(tenant_id, queja_id, filename):
15     """Pattern: tenants/{tenantId}/quejas/{quejaId}/{uuid}-{filename}"""
16     safe_filename = re.sub(r"[^a-zA-Z0-9._-]", "_", filename)
17     file_id = uuid.uuid4().hex[:12]
18     return
19         f"tenants/{tenant_id}/quejas/{queja_id}/{file_id}-{safe_filename}"
20
21 def generate_presigned_upload_url(
22     queja_id, tenant_id, filename, content_type, file_size=None
23 ):
24     settings = get_settings()
25     bucket = settings.files_bucket_resolved
26
27     # Validar content type
28     if content_type not in ALLOWED_CONTENT_TYPES:
29         raise FileError(f"Content type {content_type} not allowed")
30
31     # Validar tama~no (si se proporcion'o)
32     if file_size is not None and file_size > MAX_FILE_SIZE_BYTES:
33         raise FileError(f"File too large: {file_size} bytes (max {MAX_FILE_SIZE_BYTES})")
34
35     s3_key = build_s3_key(tenant_id, queja_id, filename)
36     s3 = boto3.client("s3")
37     expires_in = 900 # 15 minutos
38
39     upload_url = s3.generate_presigned_url(
40         "put_object",
41         Params={
42             "Bucket": bucket,
43             "Key": s3_key,

```

```

44         "ContentType": content_type,
45     },
46     ExpiresIn=expires_in,
47 )
48
49 return {
50     "uploadUrl": upload_url,
51     "fileKey": s3_key,
52     "fileUrl": build_s3_url(bucket, s3_key),
53     "expiresIn": expires_in,
54     "maxSize": MAX_FILE_SIZE_BYTES,
55 }

```

Listing 2: src/services/file_service.py

Decisiones:

- **✓Key pattern:** tenants/{tenantId}/quejas/{quejaId}/... (multi-tenant)
- **✓Sanitize filename:** solo alfanumérico, puntos, guión
- **✓Validar ContentType:** solo PDFs, imágenes, docs
- **✓Validar tamaño:** 10MB max
- **✓Expiración:** 15 min (suficiente para upload)

4. Componente 3: upload_url Lambda

4.1. El handler

```

1 from pydantic import BaseModel, Field
2
3 class UploadURLRequest(BaseModel):
4     filename: str = Field(min_length=1, max_length=255)
5     contentType: str = Field(min_length=1, max_length=100)
6     fileSize: int | None = Field(default=None, ge=0, le=10_485_760)
7
8 @error_handler
9 def lambda_handler(event, context):
10     with_correlation_id(event)
11     tenant_id = extract_tenant_id(event)
12     log.append_keys(tenant_id=tenant_id)
13
14     # Extraer quejaId del path
15     path_params = event.get("pathParameters") or {}
16     queja_id = path_params.get("quejaId") or path_params.get("id")
17     if not queja_id:
18         raise ValidationError(message="Missing quejaId in path")
19
20     # Verificar que la queja existe (multi-tenant safety)
21     dynamo = get_dynamo_client()
22     dynamo.get_queja(tenant_id=tenant_id, queja_id=queja_id) # raises
23     404
24
25     # Parse + validate body
26     body = _parse_body(event)
27     req = UploadURLRequest.model_validate(body)

```

```

28     # Generar presigned URL
29     result = generate_presigned_upload_url(
30         queja_id=queja_id,
31         tenant_id=tenant_id,
32         filename=req.filename,
33         content_type=req.contentType,
34         file_size=req.fileSize,
35     )
36
37     return HttpResponse.ok(result)

```

Listing 3: src/handlers/upload_url.py

Patrones importantes:

- ✓ Verifica que la queja existe antes de generar URL (404 si no)
- ✓ Valida con Pydantic (ContentType, FileSize, etc.)
- ✓ tenantId del header, no del body (seguridad)

5. Componente 4: file_processor Lambda (S3 event consumer)**5.1. El handler**

```

1  import re
2  from typing import Any
3
4  def _parse_key(s3_key):
5      """Extrae tenant_id y queja_id del S3 key."""
6      match = re.match(r"^tenants/([~/]+)/quejas/([~/]+)/", s3_key)
7      if not match:
8          return None
9      return match.group(1), match.group(2)
10
11
12 def _process_record(record):
13     """Procesa un solo record de S3."""
14     bucket = record["s3"]["bucket"]["name"]
15     s3_key = record["s3"]["object"]["key"]
16
17     parsed = _parse_key(s3_key)
18     if not parsed:
19         log.warning("Could not parse S3 key")
20         return
21     tenant_id, queja_id = parsed
22
23     # Construir URL p\ublica del archivo
24     file_url = build_s3_url(bucket, s3_key)
25
26     # Actualizar la queja con esta URL (append a adjuntos)
27     dynamo = get_dynamo_client()
28     item = dynamo.get_queja(tenant_id=tenant_id, queja_id=queja_id)
29     adjuntos = item.get("adjuntos", []) or []
30     if file_url not in adjuntos:
31         adjuntos.append(file_url)
32     dynamo.update_item(
33         pk=queja_pk(tenant_id, queja_id),
34         sk=queja_sk(queja_id),

```

```

35         updates={"adjuntos": adjuntos, "updatedAt": now_iso()},
36     )
37
38
39 @error_handler
40 def lambda_handler(event, context):
41     for record in event.get("Records", []):
42         try:
43             _process_record(record)
44         except Exception:
45             log.exception("Failed to process S3 record")
46     return {"statusCode": 200, "body": "OK"}

```

Listing 4: src/handlers/file_processor.py

Patrones:

- **✓ Parse S3 key:** tenants/{tenantId}/quejas/{quejaId}/... → extrae ambos IDs
- **✓ Append, not overwrite:** si el archivo ya está, no duplica
- **✓ Continue on error:** si un record falla, sigue con los demás

6. Componente 5: SAM template (los Lambdas nuevos)

6.1. Upload URL Lambda + permisos

```

1 UploadURLFunction:
2   Type: AWS::Serverless::Function
3   Properties:
4     Role: !Ref LambdaRoleArn
5     FunctionName: !Sub "sentinel-upload-url-${Stage}"
6     CodeUri: ../src
7     Handler: handlers.upload_url.lambda_handler
8     Environment:
9       Variables:
10        DYNAMODB_TABLE: !Sub "sentinel-quejas-${Stage}"
11        FILES_BUCKET: !Ref FilesBucket
12     Events:
13       UploadApi:
14         Type: Api
15         Properties:
16           Path: /api/quejas/{quejaId}/upload-url
17           Method: POST
18           RestApiId: !Ref SentinelApi
19     Policies:
20       - S3ReadPolicy:
21         BucketName: !Ref FilesBucket
22       - Statement:
23         - Effect: Allow
24           Action: s3:PutObject
25           Resource: !Sub "arn:aws:s3:::${FilesBucket}/*"

```

Listing 5: infra/template.yaml UploadURLFunction

Permisos necesarios:

- **✓ S3ReadPolicy:** para verificar si el objeto existe (opcional)

- ✓ `s3:PutObject`: para generar presigned URL (necesario)
- ✗ `No s3:*`: principio de menor privilegio

7. El problema: circular dependency

7.1. El bug que enfrentamos

Al intentar agregar el S3 event al Lambda, obtuvimos:

```

1 Error: Circular dependency between resources:
2   [SentinelApi, FileProcessorFunction,
3     CreateQuejaFunctionCreateApiPermissionStage,
4     FilesBucket, ListQuejasFunctionListApiPermissionStage,
   UploadURLFunction,
   FileProcessorFunctionS3EventPermission, ...]
```

7.2. ¿Por qué?

El problema:

- **FilesBucket** (con NotificationConfiguration) **necesita** el ARN de FileProcessorFunction
- **FileProcessorFunction** **depende** de FilesBucket (para permisos)
- → **dependencia circular**

7.3. La solución: configurar S3 notification manual

En vez de usar SAM event source (que crea el notification automáticamente), definimos el Lambda Permission por separado y configuramos la notificación con AWS CLI post-deploy.

```

1 FileProcessorFunction:
2   Type: AWS::Serverless::Function
3   Properties:
4     Role: !Ref LambdaRoleArn
5     FunctionName: !Sub "sentinel-file-processor-${Stage}"
6     CodeUri: ../src
7     Handler: handlers.file_processor.lambda_handler
8     Environment:
9       Variables:
10        DYNAMODB_TABLE: !Sub "sentinel-quejas-${Stage}"
11        FILES_BUCKET: !Ref FilesBucket
12        # SIN S3 event aqui (causa circular dep)
13      Policies:
14        - S3ReadPolicy:
15            BucketName: !Ref FilesBucket
16        - DynamoDBCrudPolicy:
17            TableName: !Ref QuejasTable
18
19 FileProcessorS3Permission:
20   Type: AWS::Lambda::Permission
21   Properties:
22     FunctionName: !Ref FileProcessorFunction.Arn
23     Action: lambda:InvokeFunction
24     Principal: s3.amazonaws.com
25     SourceAccount: !Ref AWS::AccountId
26     SourceArn: !Sub "arn:aws:s3:::${FilesBucket}"
```


Listing 6: infra/template.yaml permission separada

```

1 BUCKET=$(aws cloudformation describe-stacks \
2   --stack-name sentinel-academia-dev \
3   --query
4     'Stacks[0].Outputs[?OutputKey=='FilesBucketName'].OutputValue' \
5   --output text)
6
7 LAMBDA_ARN=$(aws lambda get-function \
8   --function-name sentinel-file-processor-dev \
9   --query 'Configuration.FunctionArn' --output text)
10
11 aws s3api put-bucket-notification-configuration \
12   --bucket "$BUCKET" \
13   --notification-configuration "{
14     \"LambdaFunctionConfigurations\": [{
15       \"Id\": \"FileProcessorTrigger\",
16       \"LambdaFunctionArn\": \"$LAMBDA_ARN\",
17       \"Events\": [\"s3:ObjectCreated:*\"],
18       \"Filter\": {
19         \"Key\": {
20           \"FilterRules\": [{\"Name\": \"prefix\", \"Value\":
21             \"tenants/\"}]
22         }
23       }
24     }
25   }]"

```

Listing 7: Post-deploy: configurar S3 notification

8. Los 2 bugs que arreglamos

Bug 1: from src.handlers._shared en file_service.py

Cuando: upload_url Lambda retorna 500.

Causa: services/file_service.py tiene imports con src. prefix que ya removimos en Fase 1.

Solución: sed -i 's/from srchandlers/from handlers/g'.

```

1 # MAL
2 from src.handlers._shared.config import get_settings
3
4 # BIEN
5 from handlers._shared.config import get_settings

```

Bug 2: KeyError 'filename' en logger

Cuando: log.info() falla con KeyError: 'attempt to overwrite 'filename' in LogRecord'.

Causa: filename es palabra reservada del stdlib logging.

Solución: usar file_name en extra={}.

```

1 # MAL
2 log.info("Generated", extra={"filename": req.filename})

```

```

3
4 # BIEN
5 log.info("Generated", extra={"file_name": req.filename})

```

9. El test E2E (paso a paso con outputs)

9.1. Paso 1: Crear queja

```

1 curl -s -X POST "$API/api/quejas" \
2   -H "Content-Type: application/json" \
3   -H "X-Tenant-ID: demo-utpl" \
4   -d '{"titulo":"Prueba con archivo adjunto",
5       "descripcion":"Subimos evidencia via presigned URL",
6       "categoriaDeclarada":"INFRAESTRUCTURA",
7       "anonima":false}'

```

```

1 {
2   "quejaId": "eb00100c-db86-4a04-8aab-67f3ba357d24",
3   "status": "PENDIENTE",
4   "correlationId": "...",
5   "createdAt": "2026-06-20T14:45:15..."
6 }

```

Listing 8: Output

9.2. Paso 2: Pedir upload URL

```

1 curl -s -X POST "$API/api/quejas/eb00100c-.../upload-url" \
2   -H "Content-Type: application/json" \
3   -H "X-Tenant-ID: demo-utpl" \
4   -d '{"filename":"test.pdf","contentType":"application/pdf","fileSize":1024}'

```

```

1 {
2   "uploadUrl": "https://sentinel-files-dev-...s3.amazonaws.com/
3     tenants/demo-utpl/quejas/eb00100c-.../c69af945dba7-test.pdf
4     ?AWSAccessKeyId=...&Signature=...&Expires=1781967798",
5   "fileKey":
6     "tenants/demo-utpl/quejas/eb00100c-.../c69af945dba7-test.pdf",
7   "fileUrl": "https://sentinel-files-dev-...s3.us-east-1.amazonaws.com/
8     tenants/demo-utpl/quejas/eb00100c-.../c69af945dba7-test.pdf",
9   "expiresIn": 900,
10  "maxSize": 10485760
11 }

```

Listing 9: Output

9.3. Paso 3: Subir archivo a S3 (directo, sin API)

```

1 echo "%PDF-1.4 fake" > /tmp/test.pdf
2 curl -s -X PUT "$UPLOAD_URL" \
3   -H "Content-Type: application/pdf" \
4   --data-binary @/tmp/test.pdf \
5   -w "HTTP %{http_code}\n" -o /dev/null
6 # Esperado: HTTP 200

```

9.4. Paso 4: S3 event → file_processor → DynamoDB

```

1 sleep 5
2 curl -s "$API/api/quejas/eb00100c-..." -H "X-Tenant-ID: demo-utpl"

1 {
2   "quejaId": "eb00100c-...",
3   "status": "ANALIZADA",
4   "adjuntos": [
5     "https://sentinel-files-dev-...s3.us-east-1.amazonaws.com/
6     tenants/demo-utpl/quejas/eb00100c-.../c69af945dba7-test.pdf"
7   ]
8 }
```

Listing 10: Output esperado

10. Verificación en AWS Console

10.1. Recursos desplegados

```

1 aws cloudformation describe-stack-resources \
2   --stack-name sentinel-academia-dev \
3   --query 'StackResources[?contains(ResourceType, 'S3') ||
4     contains(ResourceType, 'Lambda')].{Type:ResourceType,
5     Name:LogicalResourceId, Status:ResourceStatus}' \
6   --output table
```

10.2. Ver logs del file_processor

```

1 aws logs tail /aws/lambda/sentinel-file-processor-dev --follow
```

Output esperado:

```

1 {"level":"INFO","location":"lambda_handler:65",
2   "message":"Processing 1 S3 records"}
3
4 {"level":"INFO","location":"_process_record:43",
5   "message":"File uploaded: tenants/.../c69af945dba7-test.pdf"}
6
7 {"level":"INFO","location":"_process_record:55",
8   "message":"Queja updated with new file URL"}
```

11. Catálogo de errores y soluciones

Error 1: Circular dependency

Cuando: sam deploy con S3 event source.

Causa: bucket y lambda se referencian mutuamente.

Solución: configurar notification con AWS CLI post-deploy.

Error 2: No module named 'src'

Cuando: from src.handlers._shared.X en servicios.

Causa: en Fase 1 quitamos el prefijo src., pero services no se actualizó.

Solución: sed para todos los archivos en `src/services/`.

Error 3: `KeyError 'filename'` en logger

Cuando: `log.info(extra={"filename": ...})`.

Causa: `filename` es palabra reservada en `stdlib logging`.

Solución: usar `file_name` o `error_message`.

Error 4: 403 al hacer PUT al presigned URL

Cuando: el cliente intenta subir y recibe 403.

Causa: la URL expiró o el `ContentType` no coincide.

Solución: pedir nueva URL con mismo `ContentType`.

Para prod: regenerar URL automáticamente en el frontend.

Error 5: `file_processor` no actualiza la queja

Cuando: el upload a S3 funciona pero la queja sigue con `adjuntos: []`.

Causa: el Lambda no se está disparando (S3 notification mal configurada).

Solución: verificar con `aws s3api get-bucket-notification-configuration`.

12. Lo que aprendimos (resumen ejecutivo)

Checklist Fase 4

1. **Presigned URLs:** cliente sube directo a S3, no a la API
2. **Expiración:** 15 min (suficiente para upload)
3. **Key pattern multi-tenant:** `tenants/{tid}/quejas/{jid}/...`
4. **ContentType whitelisting:** solo PDFs, imágenes, docs
5. **Max size 10MB:** validado en backend antes de generar URL
6. **S3 event → file_processor:** desacopla upload de update
7. **Circular dep:** usar permission manual + post-deploy CLI
8. **file_name en logger:** `filename` es palabra reservada
9. **Sanitize filename:** solo alfanumérico, `.`, `_`, `-`
10. **Append, not overwrite:** si el archivo ya está, no duplica

13. Ejercicios para practicar

13.1. Ejercicio 1: GET `/api/quejas/{id}/files`

Endpoint que lista los archivos adjuntos de una queja. Usa `list_queja_files` que ya existe en `file_service`.

13.2. Ejercicio 2: DELETE `/api/quejas/{id}/files/{key}`

Eliminar un archivo de S3 y de los adjuntos. Requiere verificar permisos.

13.3. Ejercicio 3: Image thumbnails

Si `contentType` es `image/*`, generar thumbnail y guardarlo en otro key. Usa `Pillow` (pero ojo con `Lambda size limit`).

13.4. Ejercicio 4: Email notification (Fase 4 parte 2)

Cuando `file_processor` termina, enviar email vía `SES` al admin del tenant. Requiere `SES sandbox` configurado.

13.5. Ejercicio 5: Antivirus scan

Agregar un `Lambda` que escanea archivos subidos con `ClamAV` antes de marcarlos como adjuntos. Patrón: `S3 event` → `scan` → `tag` → `notify`.

14. Siguiendo paso: Fase 5 (RAG)

En la **Fase 5** agregamos RAG (Retrieval Augmented Generation):

- Documentación del reglamento universitario
- Embeddings con Cohere `embed-multilingual-v3.0`
- Vector store (in-memory o FAISS)
- `Lambda chat` que toma una pregunta, busca docs relevantes, llama Cohere con contexto

Tiempo estimado: 2-3 horas.

Fase	Estado
0. Setup + OCI	[OK]
1. Backend Core	[OK]
2. LLM integration	[OK]
3. Frontend deploy	[OK]
4. Files (este tutorial)	[OK]
5. RAG	próximo
6. Testing	pendiente
7. Deploy final + demo	pendiente

Fin del tutorial. A por la Fase 5!